INSTITUTO TECNOLÓGICO SUPERIOR TECSUP

DISEÑO Y DESARROLLO DE SOFTWARE

CURSO: DESARROLLO DE SOLUCIONES EN LA NUBE

PROYECTO:

LeGrandHotel – Sistema web de gestión de hospedaje

INFORME DE PRUEBAS DE CARGA CON K6

AREQUIPA, 2026

Docente:

VÍCTOR CALVO

Presentado por:

- Huacca Ccaso, Diego Armando
- Marcani Leyva, Joseph Fredy
- Pinto Garate, Leo Mathiu
- Quispe Encarnación, Rusell

Arequipa – Perú

2026

Índice

1. Qué es k6 y para qué lo usamos 3
2. Dónde se hicieron las pruebas 3
3. Preparación 4
4. El código de las pruebas 4
4.1 Configuración compartida (lib/config.js) 5
4.2 Prueba de humo (smoke.js) 5
4.3 Límite de login (rate-limit.js) 6
4.4 Escenario de recepción (reservas.js) 7
5. Prueba 1: humo (smoke) 8
5.1 Cómo leer la respuesta 9
6. Prueba 2: límite de intentos de login 10
7. Pruebas 3 a 6: carga por escalones 11
7.1 Escalón de 5 usuarios 12
7.2 Escalones de 10, 25 y 50 usuarios 14
7.3 Comparación de los cuatro escalones 16
8. Reporte HTML con gráficos 18
9. Glosario: qué significa cada dato de k6 18
10. Conclusiones 19
Anexo A. Cómo repetir las pruebas 21
Anexo B. Archivos 21

1. Qué es k6 y para qué lo usamos

k6 es una herramienta de Grafana para pruebas de carga. Se escribe un guion (script) en JavaScript que dice qué peticiones hacer a la API; k6 lo ejecuta con muchos «usuarios virtuales» (VU) a la vez, mide cuánto tarda cada respuesta y al final muestra un resumen. Un usuario virtual es un programa que repite el guion una y otra vez como si fuera una persona usando el sistema.

Con k6 respondemos tres preguntas sobre LeGrandHotel:

- **¿Funciona?** Prueba de humo (smoke): pocos usuarios, solo para ver que la API, la base de datos y la caché responden bien.
- **¿Se protege?** Prueba del límite de login: después de 5 intentos por minuto la API debe responder 429.
- **¿Cuánto aguanta?** Pruebas por escalones: el mismo trabajo de recepción con 5, 10, 25 y 50 usuarios a la vez, para ver cómo cambia el tiempo de respuesta.

Cada guion tiene «umbrales» (thresholds): metas que, si no se cumplen, hacen que k6 marque la corrida como fallida (✗) y termine con código de salida 99. Así la prueba se aprueba o se reprueba sola, sin interpretar a ojo.

2. Dónde se hicieron las pruebas

Todas las corridas de este informe se hicieron el 27 de octubre de 2026 en la laptop del equipo, contra la API corriendo en la misma máquina. No se usó AWS (la sección 3.3 del informe principal sigue pendiente).

Dato	Valor
Equipo	Laptop del equipo · Windows 11 Pro
Herramienta	k6 v2.2.0 (instalado con winget)
API	Laravel 12 sobre el servidor integrado de PHP 8.2 (php -S): atiende una petición a la vez
Base de datos	SQLite con los datos masivos: 70 habitaciones, 1000 huéspedes y 5000 reservas
Usuarios de prueba	50 recepcionistas k6-0 … k6-49 (LoadTestUsersSeeder)
Dirección	http://[::1]:8000/api/v1 (se usa [::1] porque el servidor de PHP escucha en IPv6)
Límites	En el puerto 8000 los límites de peticiones están altos para que los 429 no cuenten como error; en el 8001 se dejó el límite real de login (5 por minuto) para la prueba del 429

Importante: el servidor integrado de PHP es solo para desarrollo. Los números de este informe muestran cómo se comporta en la laptop, no la capacidad final del hotel, que se medirá en AWS con Nginx y PHP-FPM.

3. Preparación

Antes de correr k6 se comprueba que k6 esté instalado y que la API esté sana. El primer comando muestra la versión de k6; el segundo llama a /health, que responde si la aplicación, la base de datos y la caché están bien y cuánto tardó cada una.



Figura 1. Comando k6 version y su respuesta.

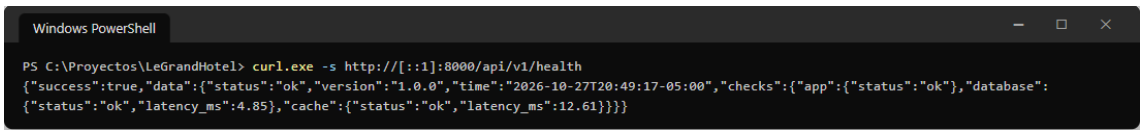


Figura 2. Comprobación de salud de la API: status «ok» en app, base de datos y caché.

Campo de la respuesta	Qué significa
success: true	Formato único de respuesta de la API: la petición salió bien
status: "ok"	Estado general de la API
checks.database	La API pudo consultar la base de datos; latency_ms es cuánto tardó en milisegundos
checks.cache	La API pudo escribir y leer en la caché
version / time	Versión desplegada y hora del servidor

Los comandos para levantar la API con estos datos están en el anexo A.

4. El código de las pruebas

Los guiones están en la carpeta tests/k6 del repositorio. Estas son las partes que se ejecutaron y qué hace cada una.

4.1 Configuración compartida (lib/config.js)

tests\k6\lib\config.js

```
// Configuración compartida de los escenarios k6 de LeGrandHotel.
import http from 'k6/http'
import { check, fail } from 'k6'

export const BASE_URL = (__ENV.BASE_URL || 'http://localhost:8000/api/v1').replace(/\/$/, '')

// Cuentas de prueba verificadas creadas con: php artisan db:seed --class=LoadTestUsersSeeder
// (usa una BD de pruebas, nunca producción con datos reales).
export const PASSWORD = __ENV.K6_PASSWORD || 'CargaK6-2026'

export const JSON_HEADERS = { 'Content-Type': 'application/json', Accept: 'application/json' }

export function auth(token) {
  return { headers: { ...JSON_HEADERS, Authorization: `Bearer ${token}` } }
}

/** Inicia sesión con los N recepcionistas de carga (LoadTestUsersSeeder) y devuelve sus tokens. */
export function createUsers(count) {
  const tokens = []
  for (let i = 0; i < count; i++) {
    const email = `k6-${i}@carga.legrandhotel.test`
    const res = http.post(`${BASE_URL}/auth/login`, JSON.stringify({ email, password: PASSWORD }), { headers: JSON_HEADERS })
    if (res.status !== 200) {
      fail(`No se pudo iniciar sesión con ${email} (${res.status}). Corre LoadTestUsersSeeder con K6_USERS >= ${count} y sube
RATE_LIMIT_LOGIN.`)
    }
    tokens.push(res.json('data.token'))
  }
  return tokens
}

export function isoDate(daysFromToday) {
```

Figura 3. tests/k6/lib/config.js: dirección de la API, inicio de sesión de los usuarios de prueba y verificación de respuestas.

Parte del código	Qué hace
BASE_URL	Dirección de la API. Se cambia con -e BASE_URL=... al correr k6
createUsers(count)	Inicia sesión con los recepcionistas de prueba y guarda sus tokens; si alguno falla, detiene la prueba con un mensaje claro
auth(token)	Arma las cabeceras con el token (Authorization: Bearer ...) para las rutas protegidas
ok(res, name)	Un check: comprueba que la respuesta sea 2xx y lo cuenta en el resumen

4.2 Prueba de humo (smoke.js)

tests\k6\smoke.js

```
import { check, sleep } from 'k6'
import { BASE_URL, JSON_HEADERS } from './lib/config.js'

export const options = {
  vus: Number(__ENV.VUS || 20),
  duration: __ENV.DURATION || '1m',
  thresholds: {
    http_req_failed: ['rate<0.01'],
    http_req_duration: ['p(95)<800'],
    checks: ['rate>0.99'],
  },
}

export default function () {
  const health = http.get(`${BASE_URL}/health`)
  check(health, {
    'health 200': (r) => r.status === 200,
    'BD ok': (r) => r.json('data.checks.database.status') === 'ok',
    'caché ok': (r) => r.json('data.checks.cache.status') === 'ok',
  })

  // Rutas protegidas responden 401 en JSON (no HTML, no 500).
  const anon = http.get(`${BASE_URL}/reservations`, { headers: JSON_HEADERS, responseCallback: http.expectedStatuses(401) })
  check(anon, { 'sin token → 401 JSON': (r) => r.status === 401 && r.json('error.code') === 'UNAUTHENTICATED' })

  sleep(1)
```

Figura 4. tests/k6/smoke.js.

Parte del código	Qué hace
vus / duration	Cuántos usuarios virtuales y por cuánto tiempo (en esta corrida: 5 VU durante 30 s)
thresholds	Metas: menos de 1 % de errores, 95 % de respuestas en menos de 800 ms y más de 99 % de checks correctos
http.get(.../health)	Pide la salud de la API
check(...)	Verifica que /health dé 200 y que la base de datos y la caché estén «ok»
/reservations sin token	Comprueba que una ruta protegida responda 401 en JSON (no una página HTML ni un error 500)
sleep(1)	Espera 1 segundo entre vueltas, como haría una persona

4.3 Límite de login (rate-limit.js)

tests\k6\rate-limit.js

```
import http from 'k6/http'
import { check } from 'k6'
import { BASE_URL, JSON_HEADERS } from './lib/config.js'

export const options = { vus: 1, iterations: 1 }

export default function () {
  const body = JSON.stringify({ email: `limite-${Date.now()}@carga.legrandhotel.test`, password: 'incorrecta' })
  const statuses = []
  for (let i = 0; i < 6; i++) {
    const res = http.post(`${BASE_URL}/auth/login`, body, { headers: JSON_HEADERS, responseCallback: http.expectedStatuses(401, 429) })
    statuses.push(res.status)
    if (i === 5) {
      check(res, {
        '6.º intento → 429': (r) => r.status === 429,
        'trae Retry-After': (r) => !!r.headers['Retry-After'],
        'formato JSON único': (r) => r.json('error.code') === 'TOO_MANY_REQUESTS',
      })
    }
  }
  console.log(`Estados recibidos: ${statuses.join(', ')}`)
}

```

Figura 5. tests/k6/rate-limit.js.

Un solo usuario intenta iniciar sesión 6 veces seguidas con una contraseña incorrecta. Los 5 primeros intentos deben dar 401 (credenciales incorrectas) y el sexto 429 (demasiados intentos), con la cabecera Retry-After que dice cuántos segundos esperar. responseCallback marca 401 y 429 como respuestas esperadas para que no cuenten como errores.

4.4 Escenario de recepción (reservas.js)

tests\k6\reservas.js

```
22
23  const PROFILE = __ENV.PROFILE || 'carga-25'
24  const VUS = Number(__ENV.VUS || 0)
25  const USERS = Number(__ENV.USERS || 50)
26
27  const profiles = {
28    // Un escalón de VUS constantes (para comparar varias corridas cortas: 5, 10, 25, 50 VU…).
29    escalon: {
30      stages: [
31        { duration: '10s', target: VUS || 10 },
32        { duration: __ENV.DURATION || '45s', target: VUS || 10 },
33        { duration: '5s', target: 0 },
34      ],
35      thresholds: { http_req_duration: ['p(95)<500'], http_req_failed: ['rate<0.01'] },
36    },
37    'carga-25': {
38      stages: [
39        { duration: '1m', target: 50 },
40        { duration: '2m', target: 50 },
41        { duration: '1m', target: 100 },
42        { duration: '2m', target: 100 },
43        { duration: '30s', target: 0 },
44      ],
45      thresholds: { http_req_duration: ['p(95)<500'], http_req_failed: ['rate<0.001'] },
```

Figura 6. tests/k6/reservas.js: perfil «escalon» y opciones.

Parte del código	Qué hace
PROFILE=escalon	Sube en 10 s hasta VUS usuarios, los mantiene 45 s y baja en 5 s (1 minuto en total)
thresholds	Metas: p95 < 500 ms y menos de 1 % de errores. Las lecturas, aparte, p95 < 800 ms
summaryTrendStats	Qué estadísticas mostrar de los tiempos: promedio, mínimo, mediana, p90, p95, p99 y máximo
reserva_creada_ms	Métrica propia: cuánto tarda crear una reserva

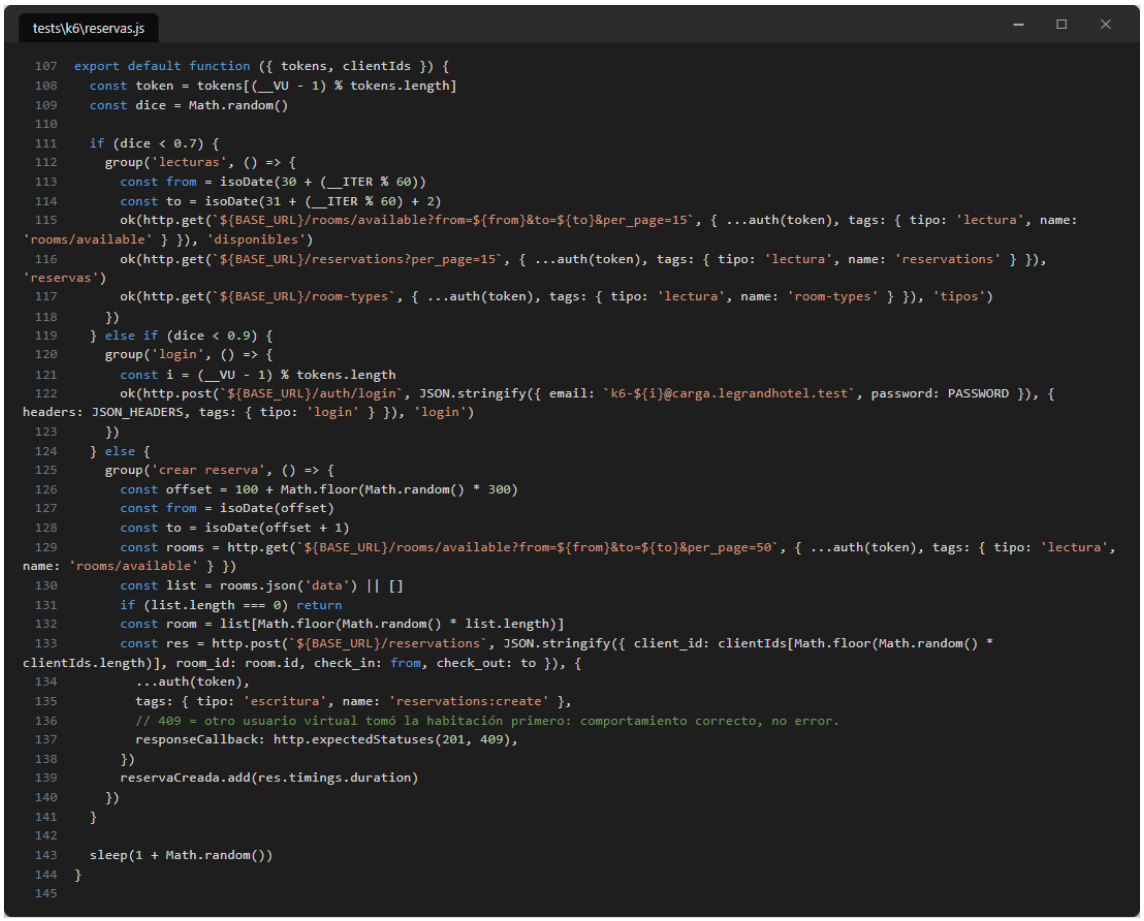


Figura 7. tests/k6/reservas.js: lo que hace cada usuario virtual en cada vuelta.

Parte del código	Qué hace
dice < 0.7 → lecturas (70 %)	Consulta habitaciones disponibles en un rango de fechas, la lista de reservas y los tipos de habitación
dice < 0.9 → login (20 %)	Vuelve a iniciar sesión con su usuario
resto → crear reserva (10 %)	Busca una habitación libre y crea una reserva a nombre de un huésped al azar
expectedStatuses(201, 409)	Si dos usuarios toman la misma habitación, el segundo recibe 409; es el comportamiento correcto y no cuenta como error
tags	Etiquetan cada petición (lectura, login, escritura) para medirlas por separado
sleep(1 + random)	Pausa de 1 a 2 segundos entre vueltas

5. Prueba 1: humo (smoke)

Qué se hizo: 5 usuarios virtuales durante 30 segundos pidieron /health y una ruta protegida sin token. Las dos primeras líneas guardan el reporte HTML con gráficos; la tercera corre la prueba y guarda el resumen en JSON.

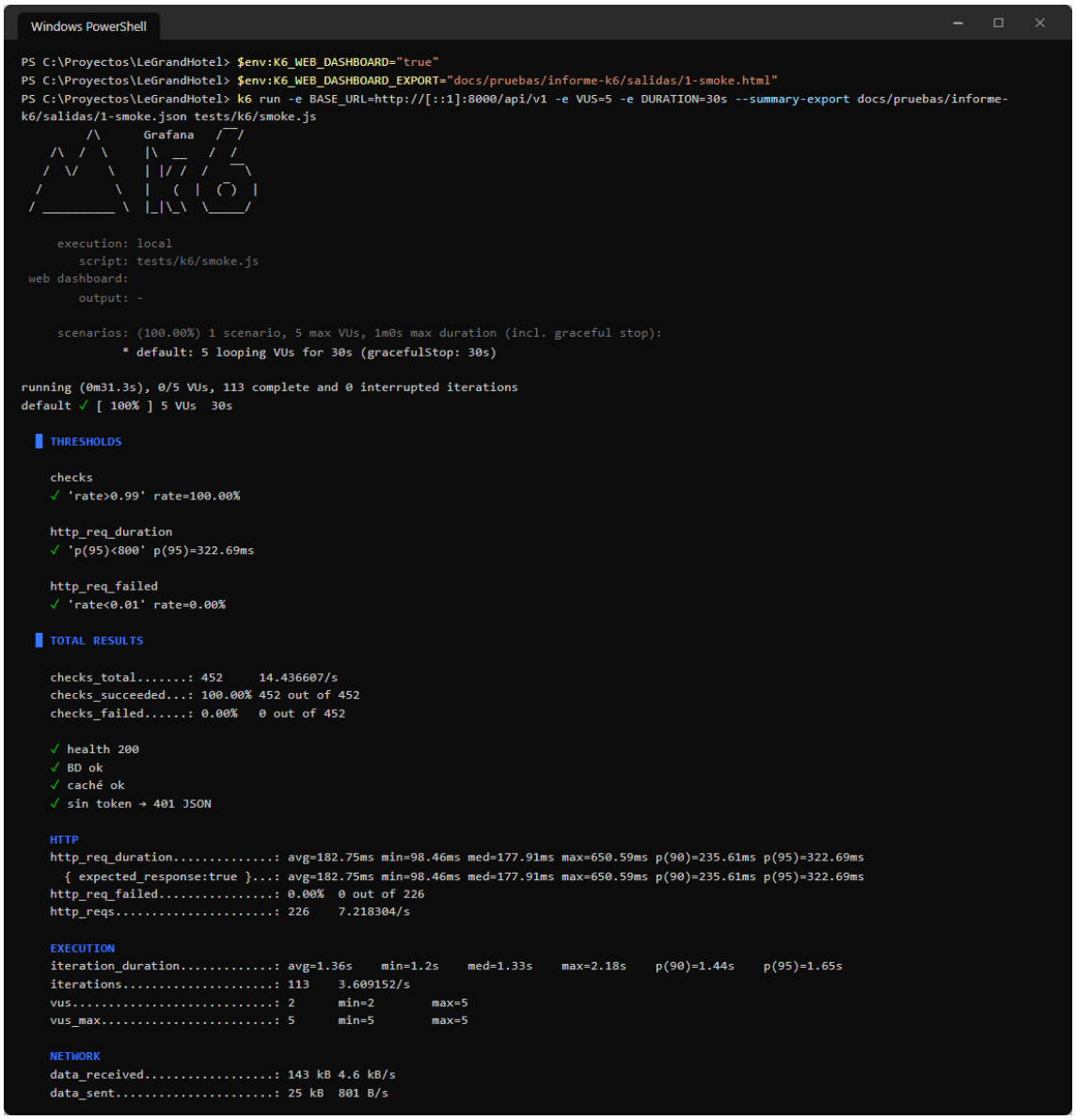


Figura 8. Comando de la prueba de humo y la respuesta de k6.

5.1 Cómo leer la respuesta

Línea de la respuesta	Qué significa	Resultado
scenarios: 1 scenario, 5 max VUs	Se corrió un escenario con hasta 5 usuarios a la vez	—
THRESHOLDS ✓ checks rate>0.99	Meta: más del 99 % de verificaciones correctas	100.00 % ✓
THRESHOLDS ✓ http_req_duration p(95)<800	Meta: el 95 % de las respuestas en menos de 800 ms	p95 = 323 ms ✓
THRESHOLDS ✓ http_req_failed rate<0.01	Meta: menos de 1 % de peticiones fallidas	0.00 % ✓
checks_succeeded	Verificaciones correctas de todas las hechas	452 de 452
✓ health 200 · BD ok · caché ok	La API, la base de datos y la caché respondieron bien en cada vuelta	todas ✓

Línea de la respuesta	Qué significa	Resultado
✓ sin token → 401 JSON	Una ruta protegida sin token responde 401 con el código UNAUTHENTICATED	✓
http_req_duration avg / med / p(95)	Tiempo de respuesta promedio, mediana y percentil 95	183 ms / 178 ms / 323 ms
http_reqs	Total de peticiones y cuántas por segundo	226 (7.2/s)
iterations	Vueltas completas del guion (cada una = 2 peticiones)	113

Conclusión: la prueba de humo pasa. La API está sana, responde rápido con pocos usuarios y las rutas protegidas no se pueden usar sin iniciar sesión.

6. Prueba 2: límite de intentos de login

Qué se hizo: un usuario intentó iniciar sesión 6 veces con una contraseña incorrecta contra la API del puerto 8001, que tiene el límite real de 5 intentos por minuto.

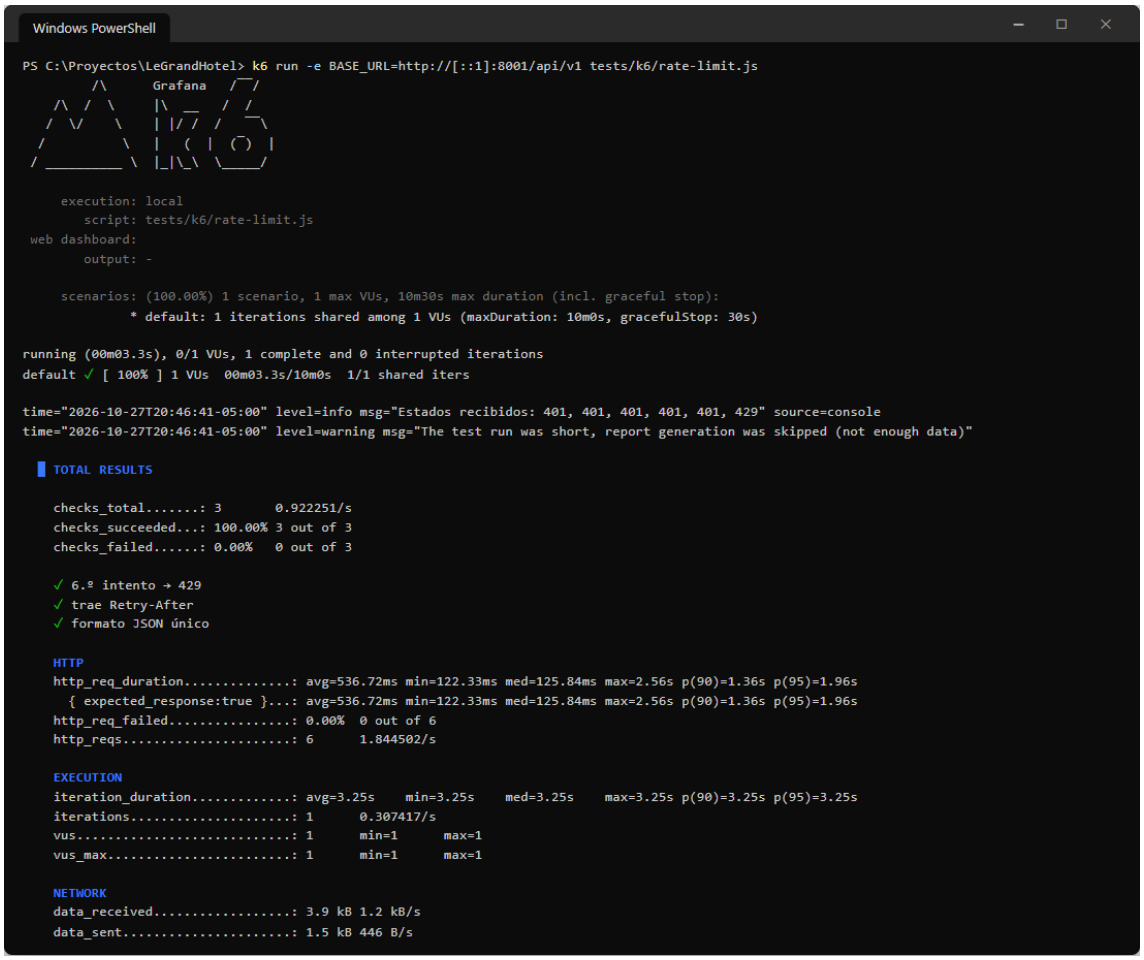


Figura 9. Comando de la prueba del límite de login y la respuesta de k6.

Línea de la respuesta	Qué significa
Estados recibidos: 401, 401, 401, 401, 401, 429	Los 5 primeros intentos dieron 401 (contraseña incorrecta) y el sexto 429 (demasiados intentos)

Línea de la respuesta	Qué significa
✓ 6.º intento → 429	La API bloqueó el sexto intento
✓ trae Retry-After	La respuesta dice cuántos segundos hay que esperar para volver a intentar
✓ formato JSON único	El error viene con el formato de la API (error.code = TOO_MANY_REQUESTS), no como página HTML
http_req_failed 0.00 %	Los 401 y 429 eran esperados, así que no cuentan como fallos
report generation was skipped	Aviso normal: la prueba duró 3 s y no hubo datos suficientes para el reporte HTML

Conclusión: el login está protegido contra ataques de fuerza bruta: nadie puede probar contraseñas sin límite.

7. Pruebas 3 a 6: carga por escalones

Qué se hizo: el escenario de recepción (70 % lecturas, 20 % login, 10 % crear reserva) se corrió cuatro veces, cada vez con más usuarios a la vez: 5, 10, 25 y 50. Cada corrida dura 1 minuto. Antes de empezar, la función setup() inicia sesión con un recepcionista de prueba por cada usuario virtual.

7.1 Escalón de 5 usuarios



Figura 10. Escalón de 5 usuarios: comando y respuesta.

Línea de la respuesta	Qué significa	Resultado
Run [100%] setup()	Terminó la preparación: inicio de sesión de los usuarios y lectura de huéspedes	—
✗ http_req_duration 'p(95)<500'	Meta de p95 menor a 500 ms. La ✗ indica que no se cumplió	p95 = 610 ms ✗
{tipo:lectura} p(95)<800	Meta aparte para las consultas (habitaciones, reservas, tipos)	p95 = 583 ms ✓
✓ http_req_failed rate<0.01	Ninguna petición falló	0.00 % ✓

Línea de la respuesta	Qué significa	Resultado
✓ disponibles · reservas · tipos · login: 2xx	Cada tipo de petición respondió bien	269 de 269
reserva_creada_ms	Cuánto tardó crear una reserva (p95)	655 ms
iteration_duration	Cuánto tardó una vuelta completa, incluida la pausa de 1–2 s	—
vus min / max	Usuarios activos: subió de 0 a 5 y bajó al final	0 → 5
data_received / data_sent	Datos que bajaron de la API y que se enviaron	—
level=error thresholds ... crossed	k6 avisa que no se cumplió una meta y sale con código 99 (prueba reprobada)	—

Con solo 5 usuarios ya no se cumple la meta de 500 ms: el p95 fue de 610 ms. No hubo errores: todas las peticiones respondieron bien, pero algunas tardaron más de lo deseado. La explicación está en la sección 7.3.

7.2 Escalones de 10, 25 y 50 usuarios



Figura 11. Escalón de 10 usuarios.



Figura 12. Escalón de 25 usuarios.



Figura 13. Escalón de 50 usuarios.

7.3 Comparación de los cuatro escalones

Usuarios	Peticiones	Pet./s	Mediana (p50)	p95	Máximo	Errores	Meta p95 < 500 ms
5	308	4.8	304 ms	610 ms	1,131 ms	0.00 %	✗ no cumple
10	320	4.9	1,155 ms	1,798 ms	2,037 ms	0.00 %	✗ no cumple
25	367	4.8	3,936 ms	4,629 ms	5,010 ms	0.00 %	✗ no cumple
50	452	4.8	7,922 ms	9,346 ms	9,823 ms	0.00 %	✗ no cumple

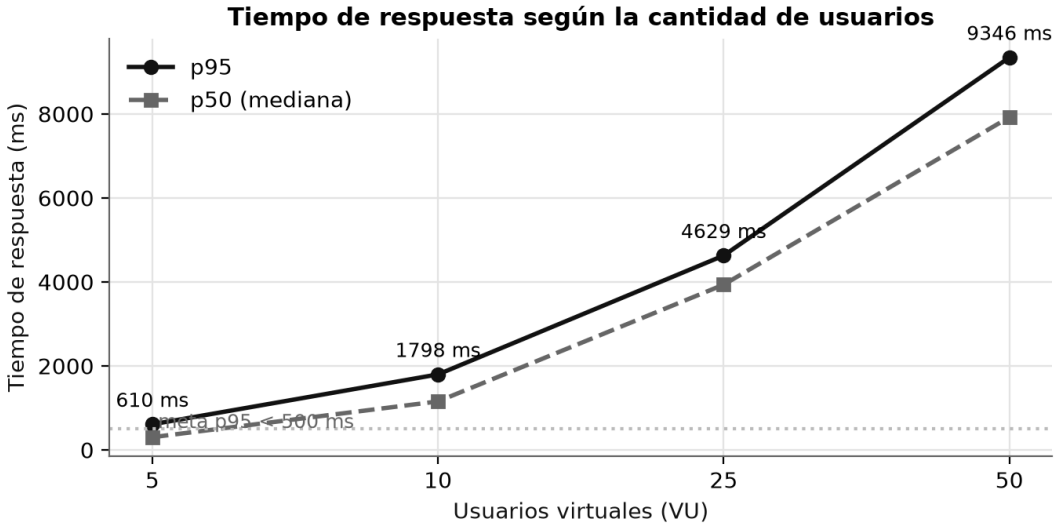


Figura 14. Mediana y p95 del tiempo de respuesta según los usuarios a la vez.

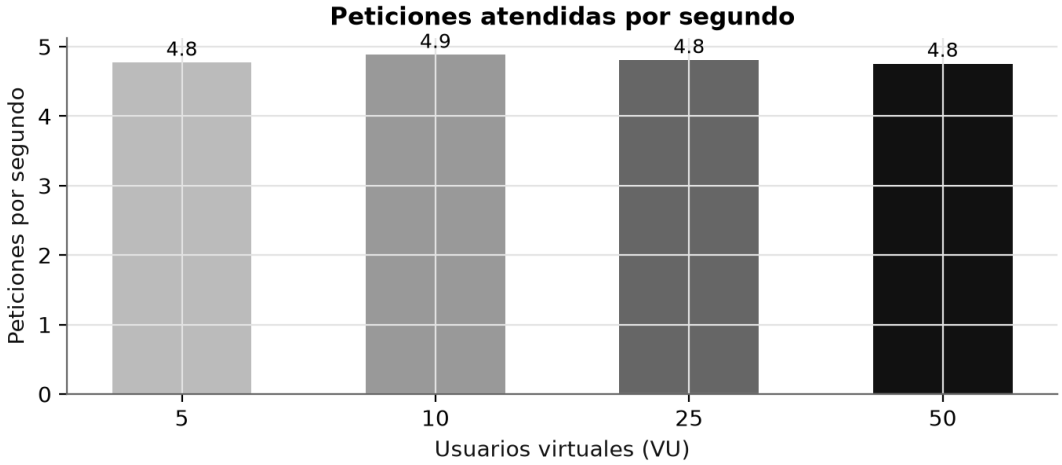


Figura 15. Peticiones por segundo que atendió la API en cada escalón.

Qué muestran los resultados:

- Las peticiones por segundo casi no suben al aumentar los usuarios (de 4.8 a 4.8): la API llegó a su techo.
- El tiempo de respuesta sí sube mucho: el p95 pasa de 610 ms con 5 usuarios a 9,346 ms con 50. Las peticiones que no se pueden atender hacen cola.
- No hubo errores en ningún escalón (0.00 % con 50 usuarios): la API responde lento, pero no se cae ni pierde datos.

Por qué pasa: el servidor integrado de PHP (php -S) atiende una sola petición a la vez. Si llegan 50 juntas, 49 esperan su turno. Por eso las peticiones por segundo se quedan fijas y el tiempo crece con la cantidad de usuarios. No es un problema del código de LeGrandHotel, sino del servidor de desarrollo.

Cómo se corrige: en AWS la API corre con Nginx y PHP-FPM (varios procesos atendiendo a la vez), OPcache, MySQL en RDS y Redis para la caché. Allí se repetirán estas mismas pruebas con 50 → 100 usuarios (entrega del 25 %) y 1000 a 5000 usuarios (entrega del 80 %).

8. Reporte HTML con gráficos

Además del resumen en la terminal, cada corrida generó un reporte HTML (variables K6_WEB_DASHBOARD y K6_WEB_DASHBOARD_EXPORT). Se abre con doble clic en el navegador y muestra cómo cambiaron los usuarios, las peticiones por segundo y los tiempos durante el minuto de prueba.

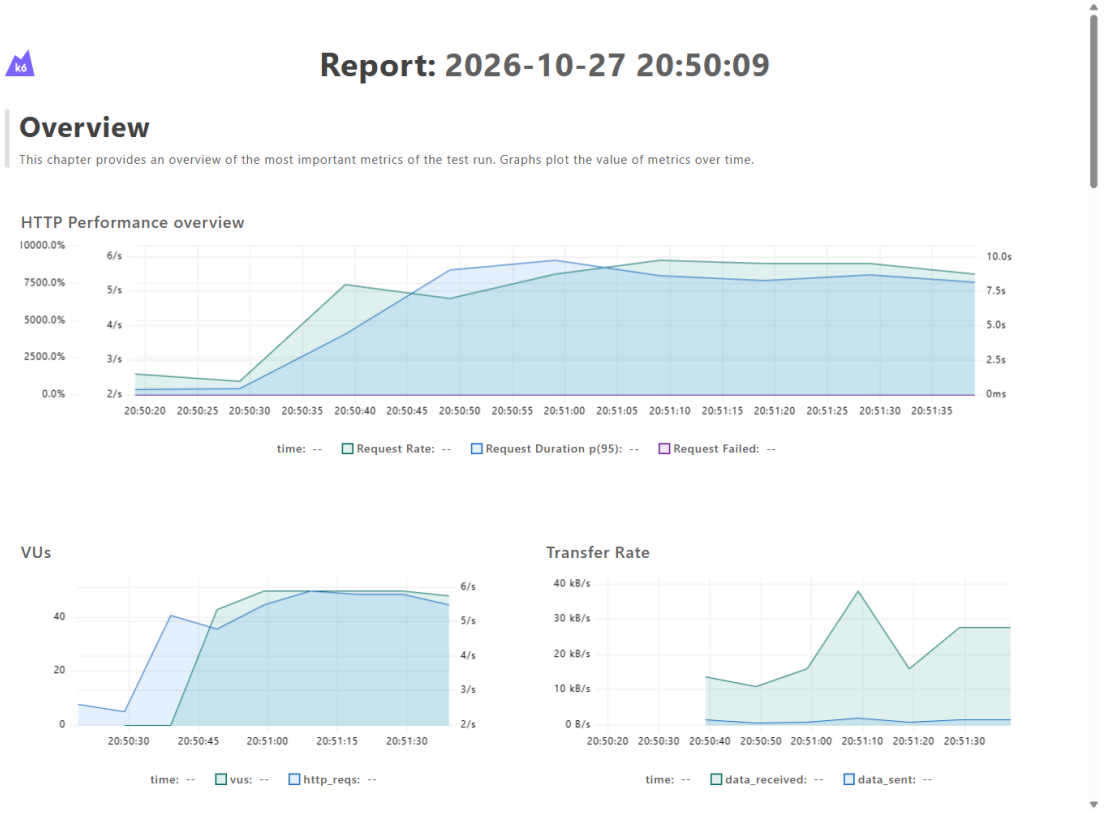


Figura 16. Reporte HTML del escalón de 50 usuarios (salidas/3-escalon-50vu.html).

Gráfico del reporte	Qué muestra
Request Rate	Peticiones por segundo durante la prueba
Request Duration	Tiempo de respuesta en cada momento (p95 y mediana)
Request Failed	Peticiones fallidas por segundo (en todas las corridas fue 0)
VUs	Usuarios virtuales activos: la subida, la meseta y la bajada del escalón

9. Glosario: qué significa cada dato de k6

Dato	Qué significa	Cómo leerlo
VU (usuario virtual)	Un programa que repite el guion como una persona	Más VU = más carga a la vez
iterations	Vueltas completas del guion	Cuánto trabajo se hizo

Dato	Qué significa	Cómo leerlo
iteration_duration	Cuánto tarda una vuelta, incluida la pausa	Sube si la API se pone lenta
http_reqs	Total de peticiones y peticiones por segundo	Capacidad: cuánto atiende la API
http_req_duration	Tiempo desde que se envía la petición hasta que llega toda la respuesta	El dato principal de velocidad
avg	Promedio de los tiempos	Se distorsiona con valores extremos
min / max	La respuesta más rápida y la más lenta	El máximo muestra el peor caso
med (p50)	Mediana: la mitad de las respuestas tardó menos que esto	La experiencia típica
p(90) / p(95) / p(99)	El 90 %, 95 % o 99 % de las respuestas tardó menos que esto	p95 es la meta habitual: casi todos los usuarios
http_req_failed	Porcentaje de peticiones con error (4xx/5xx no esperados o sin respuesta)	Debe ser 0 % o casi
expected_response:true	Los tiempos solo de las respuestas correctas	Evita que errores rápidos mejoren el promedio
checks	Verificaciones del guion (código 200, JSON correcto…)	✓ pasó · ✗ falló
thresholds	Metas de la prueba	✓ cumplida · ✗ no cumplida (código de salida 99)
{tipo:lectura}	Métrica filtrada por etiqueta: solo las lecturas	Compara tipos de petición
reserva_creada_ms	Métrica propia del proyecto: tiempo de crear una reserva	La operación más pesada
vus / vus_max	Usuarios activos en el momento y el máximo reservado	Confirma la carga aplicada
data_received / data_sent	Datos recibidos de la API y enviados a ella	Tamaño del tráfico
setup()	Preparación que corre una vez antes de la prueba	Aquí: login de los usuarios de prueba

10. Conclusiones

Prueba	Resultado	Qué nos dice
1. Humo (5 VU, 30 s)	✓ pasa · p95 323 ms · 0 % errores	La API, la base de datos y la caché funcionan; las rutas protegidas exigen sesión
2. Límite de login	✓ pasa · 429 en el 6.º intento	El login está protegido contra fuerza bruta
3. Escalón 5 VU	✗ p95 610 ms · 0.00 % errores	Sin errores; tiempo de respuesta mayor a la meta
4. Escalón 10 VU	✗ p95 1,798 ms · 0.00 % errores	Sin errores; tiempo de respuesta mayor a la meta
5. Escalón 25 VU	✗ p95 4,629 ms · 0.00 % errores	Sin errores; tiempo de respuesta mayor a la meta

Prueba	Resultado	Qué nos dice
6. Escalón 50 VU	✗ p95 9,346 ms · 0.00 % errores	Sin errores, pero el servidor de desarrollo hace cola

- La API es correcta y estable: en las 6 corridas no hubo ninguna petición fallida ni errores 500.
- La seguridad funciona bajo prueba: 401 sin token y 429 al pasar el límite de login, siempre en el formato JSON de la API.
- La velocidad no cumple la meta en la laptop porque el servidor de desarrollo de PHP atiende una petición a la vez. La corrección (PHP-FPM, OPcache, RDS y Redis) se aplicará al desplegar en AWS, donde se repetirán las pruebas con 100 y luego 1000 a 5000 usuarios.

Anexo A. Cómo repetir las pruebas

1. Levantar la API con los datos de prueba (PowerShell, desde la carpeta del proyecto):

```
cd api
php artisan migrate
php artisan db:seed --class=MassiveDataSeeder      # solo si la base está vacía
$env:K6_USERS="50"; php artisan db:seed --class=LoadTestUsersSeeder
$env:RATE_LIMIT_API="100000"; $env:RATE_LIMIT_LOGIN="100000"
php -S localhost:8000 -t public public/index.php
```

2. En otra ventana, desde la carpeta del proyecto, correr cada prueba (PowerShell):

```
k6 run -e BASE_URL=http://[::1]:8000/api/v1 -e VUS=5 -e DURATION=30s
tests/k6/smoke.js
k6 run -e BASE_URL=http://[::1]:8001/api/v1 tests/k6/rate-limit.js
k6 run -e BASE_URL=http://[::1]:8000/api/v1 -e PROFILE=escalon -e VUS=50 -e
USERS=50 tests/k6/reservas.js
```

Para el límite de login se levanta una segunda API en el puerto 8001 con el límite real (RATE_LIMIT_LOGIN=5). Para guardar el reporte HTML, antes de k6 run:

```
$env:K6_WEB_DASHBOARD="true"
$env:K6_WEB_DASHBOARD_EXPORT="docs/pruebas/informe-k6/salidas/3-escalon-
50vu.html"
```

En cmd las variables se escriben con set y comillas, por ejemplo: set "K6_WEB_DASHBOARD=true". Todas las corridas de este informe se pueden repetir de una vez con docs/pruebas/informe-k6/correr.sh (Git Bash).

Anexo B. Archivos

Archivo	Contenido
`tests/k6/*.js`	Guiones de las pruebas
`docs/pruebas/informe-k6/salidas/*.txt`	Salida completa de la terminal de cada corrida
`docs/pruebas/informe-k6/salidas/*.json`	Resumen de cada corrida en JSON (fuente de las tablas)
`docs/pruebas/informe-k6/salidas/*.html`	Reporte con gráficos de cada corrida
`docs/pruebas/informe-k6/graficos.py`	Genera los gráficos de la sección 7.3